Graph Neural Networks

and it's applications

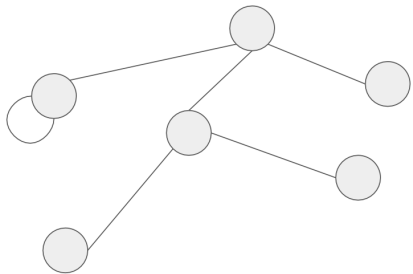
Danila Biktimirov
AST
CUB

30 April 2026

Overview

1. intro, graph types, tasks
2. statistics, graphlets, graphlet kernels, WL
3. why GNNs, message passing, GCN, GraphSAGE, GAT
4. knowledge graphs
5. where graph models win

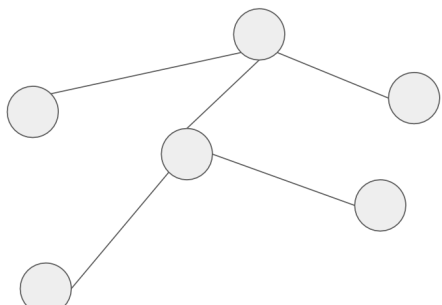
What are Graphs?



What are Graphs?

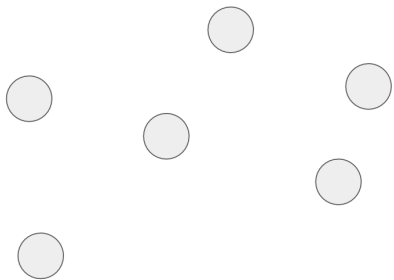
$G = (V, E)$, where:

- V is the set of vertices (or nodes), representing the objects.
- E is the set of edges, representing the relationships or connections between the vertices.



What are Graphs?

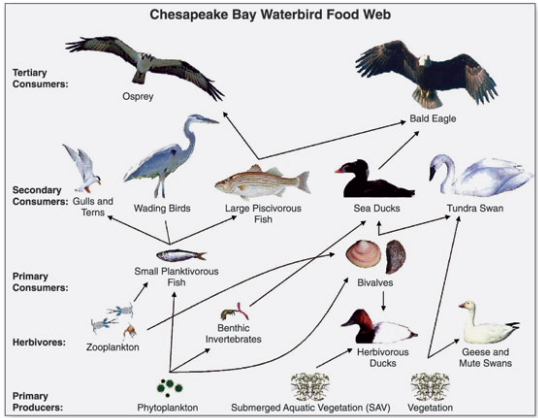
Graph?



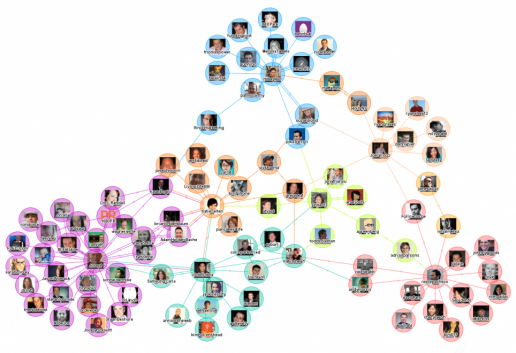
Why Graphs are Needed in the Context of Data?

- Graphs are a generalized way of representing any information related to interactions between objects.
- There are many different methods for graph analysis, making it convenient to transform data from various domains into a unified format.

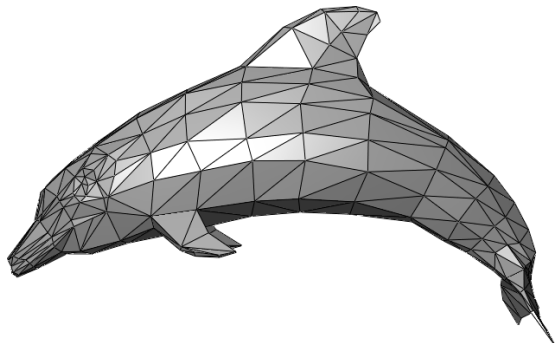
Examples of Graph Data



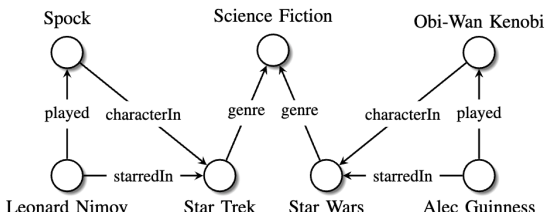
Examples of Graph Data



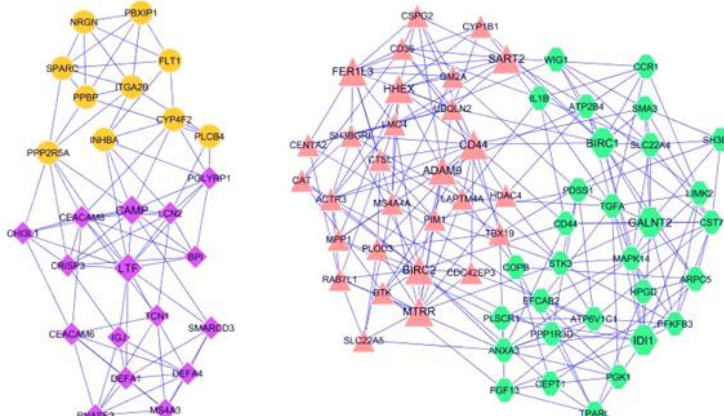
Examples of Graph Data



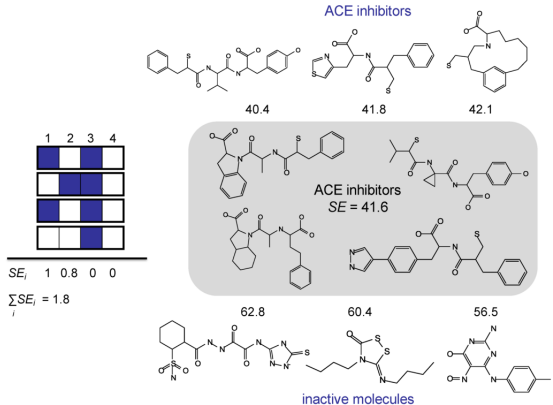
Examples of Graph Data



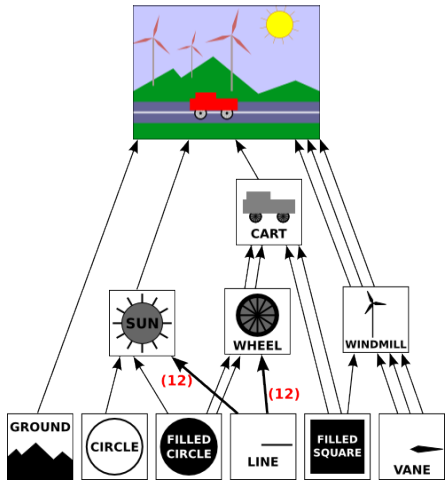
Examples of Graph Data



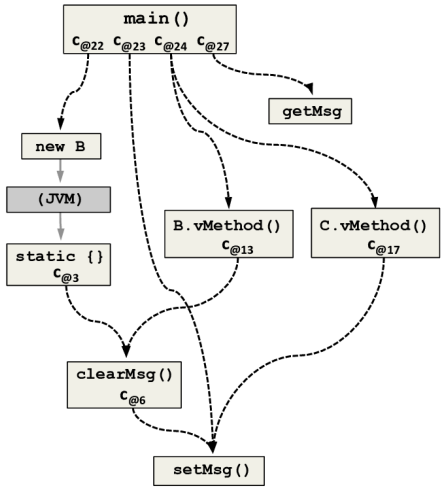
Examples of Graph Data



Examples of Graph Data



Examples of Graph Data



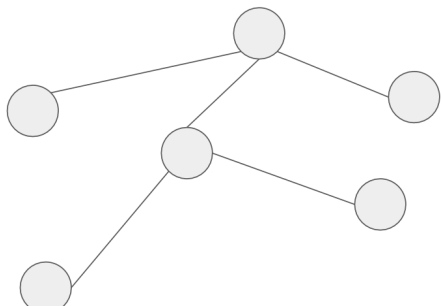
Motivation

The primary applications of deep learning today involve sequential data:

- **NLP:** "askfnkksnf" - sequences of text
- **CV:** Structured data from image pixels
- **Audio:** Sound waves

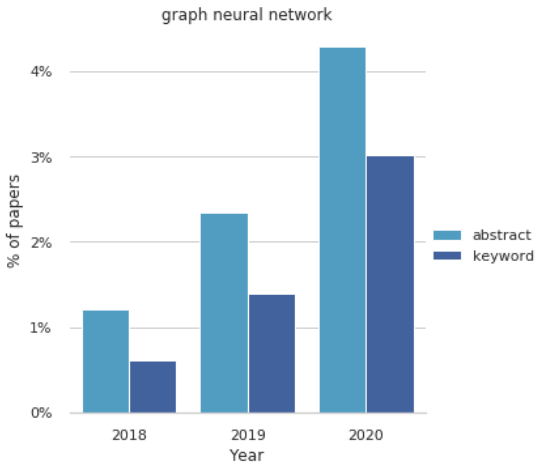
Motivation

The distinctive feature of graphs is the relationships between objects, allowing information to be processed more broadly and differently.



Where is the start and end?

Motivation


graph neural network
4%
3%
2%
1%
0%
% of papers
2018
2019
2020
Year
abstract
keyword

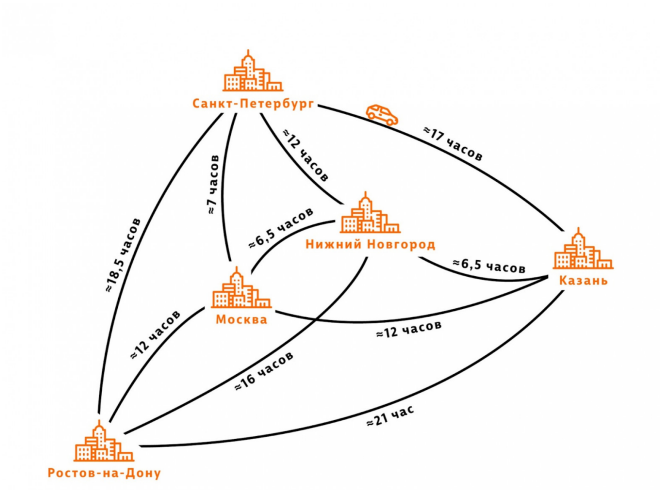
Approaches to Graph Data Analysis Without Neural Networks

Many tasks that are currently solved using deep learning have existed for a long time in the context of graph data. Correspondingly, various methods for solving these tasks using conventional graph algorithms have also existed.

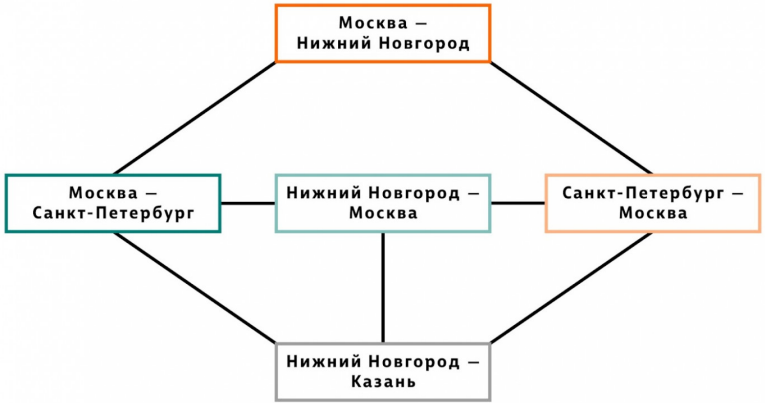
Approaches to Graph Data Analysis Without Neural Networks

- Групповые вкусы пользователя - **начальные вершины обхода графа**
- Делаем 3 случайных шага по графу
- В каждой вершине есть вероятность телепортироваться в начало обхода
- Конечные вершины графа - **кандидаты для рекомендации**

Approaches to Graph Data Analysis Without Neural Networks

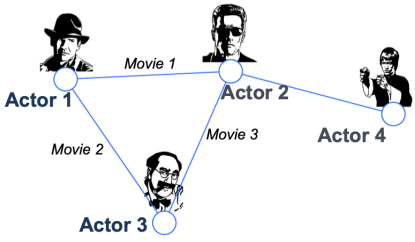


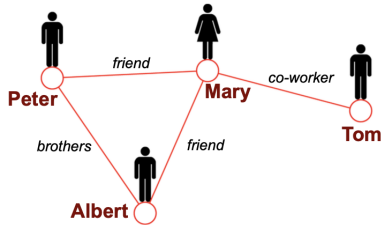
Approaches to Graph Data Analysis Without Neural Networks

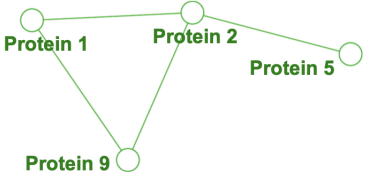


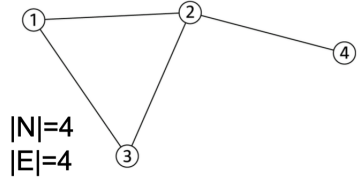
GlowByte

Finally, Graphs!









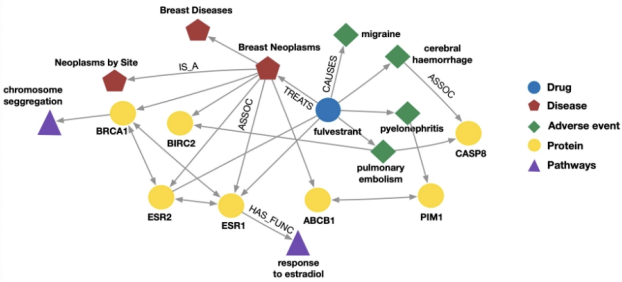
Finally, Heterographs!

A heterogeneous graph is defined as:

$$G = (V, E, R, T)$$

- **Nodes with node types:** $v_i \in V$
- **Edges with relation types:** $(v_i, r, v_j) \in E$
- **Node type:** $T(v_i)$
- **Relation type:** $r \in R$
- **Nodes and edges have attributes/features.**

Finally, Heterographs!



Biomedical Knowledge Graphs

Example node: Migraine
Example edge: (fulvestrant, Treats, Breast Neoplasms)
Example node type: Protein
Example edge type (relation): Causes

Academic Graphs

Example node: ICML
Example edge: (GraphSAGE, NeurIPS)
Example node type: Author
Example edge type (relation): pubYear

Choosing a Proper Representation

How to build a graph?

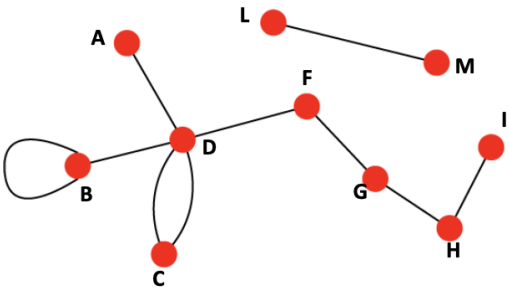
- **What are nodes?**
- **What are edges?**

Choice of the proper network representation of a given domain/problem determines our ability to use networks successfully:

- In some cases, there is a unique, unambiguous representation.
- In other cases, the representation is by no means unique.
- The way you assign links will determine the nature of the question you can study.

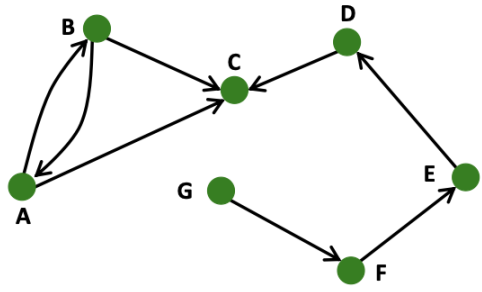
Undirected vs Directed Graphs

- **Links:** undirected (symmetrical, reciprocal)



- Weights
- Properties

- **Links:** directed

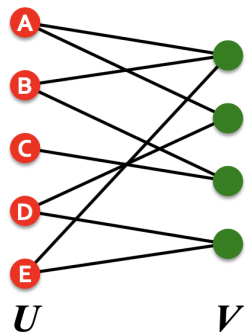


- Types
- Attributes

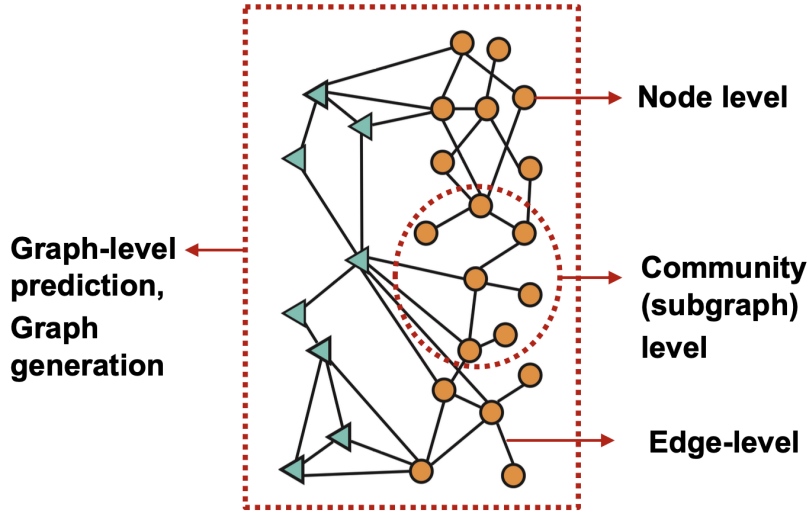
Bipartite graph

Bipartite graph is a graph whose nodes can be divided into two disjoint sets U and V such that every link connects a node in U to one in V; that is, U and V are **independent sets**.

- **Examples:**
  - Authors-to-Papers (they authored)
  - Actors-to-Movies (they appeared in)
  - Users-to-Movies (they rated)
  - Recipes-to-Ingredients (they contain)
- **"Folded" networks:**
  - Author collaboration networks
  - Movie co-rating networks



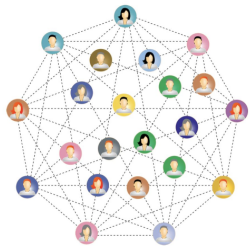
Types of Graph Tasks



Types of Graph Tasks

1. **Edge-level** – Tasks related to edges:
   - Link classification
   - Regression
   - Graph completion
2. **Node-level** – Tasks related to nodes:
   - Node classification
   - Regression
   - Clustering
3. **Graph-level** – Tasks specific to the entire graph:
   - Graph classification
   - Regression
   - Generation
   - Evolution

Node-Level Tasks



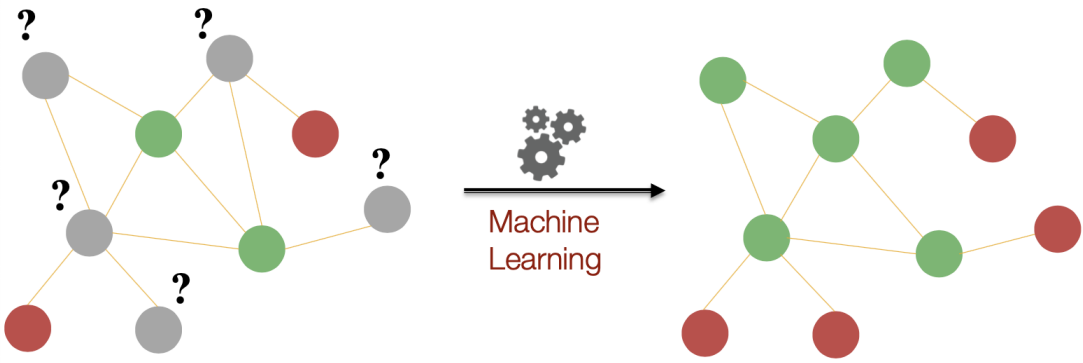
Determining a person's interests based on their surroundings

We aim to learn the function:

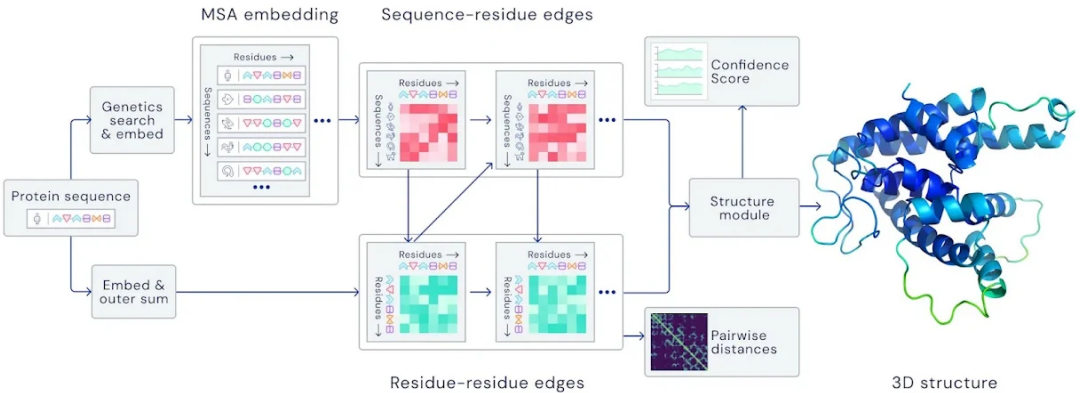
$$f : (V, E) \rightarrow \mathbb{R}^{|V|}$$

where V is the set of nodes, E is the set of edges, and the result is a numerical representation for each node in the graph.

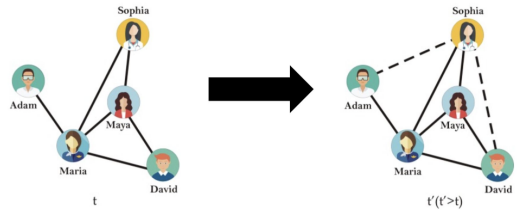
Node-Level Tasks



Node-Level Tasks



Edge-level Tasks



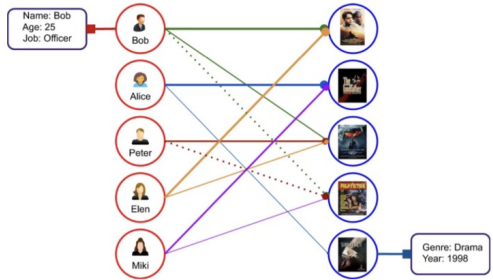
Make friend recommendations in a social network

We aim to make predictions for all pairs (v_i, v_j):

$$f(v_i, v_j)$$

where v_i and v_j are nodes in the graph.

Edge-level Tasks



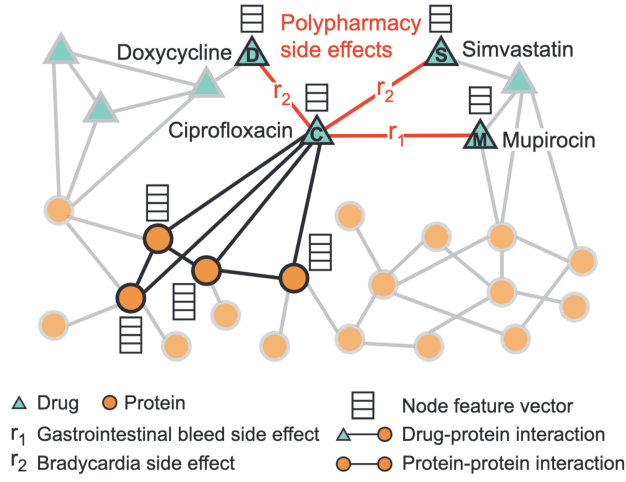
Make recommendations for movies or music

We aim to make predictions for all pairs (v_i, v_j):

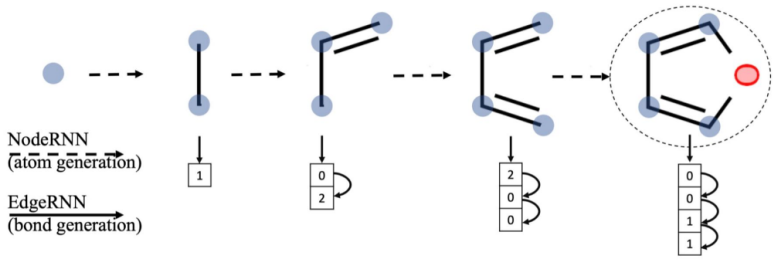
$$f(v_i, v_j)$$

where v_i represents a user and v_j represents an item (e.g., movie or music).

Edge-level Tasks

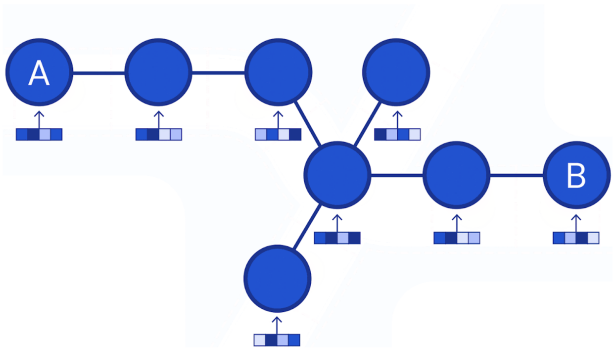


Graph-level Tasks



We aim to learn how to generate graphs with specific characteristics.

Subgraph-level Tasks

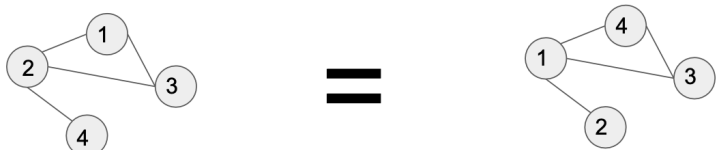


Predicting vehicle arrival time:

- The road is divided into segments (nodes).
- Accessibility between segments (edges) determines connectivity.

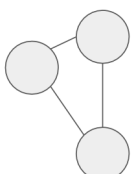
Features of Graph Algorithms

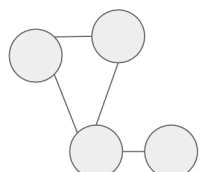
Graph algorithms must operate independently of the permutation of vertex indices.

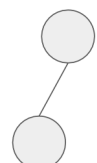


Features of Graph Algorithms

Graph algorithms must be able to handle graphs of different sizes as input.







Before Deep Learning

Before moving on to core deep learning methods, we should first explore existing approaches for creating so-called **hand-crafted features**.

By analogy with the types of existing tasks for graph data analysis, we will examine features for **nodes, edges, and graphs**.

IMPORTANT: We assume that all graphs discussed in the lecture will be **undirected**.

Problem Formulation

Task: Make predictions for a set of objects.

Features: n-dimensional vectors
Objects: Nodes, edges, graphs
Loss function: Depends on the task

Node-level

- Node degree
- Centrality
- Clustering coefficient
- Graphlets

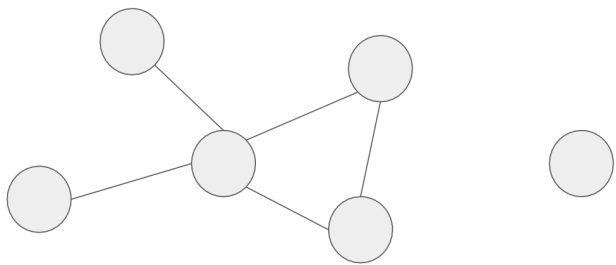
Node Degree

Definition: The degree of a node is the number of edges incident to it.

In the case of undirected graphs without loops, the node degree can be considered as the number of neighbors of that node.

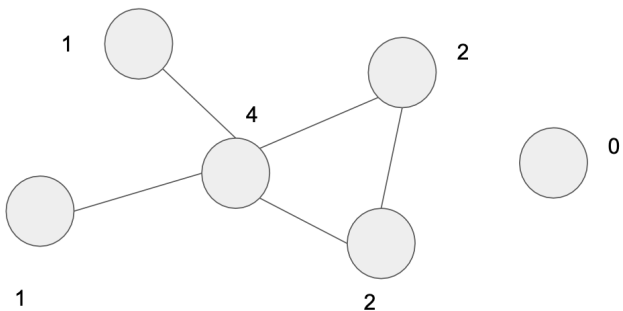
$$d_u = \sum_{v \in V} A[u, v]$$

Node Degree



Node Degree

The main property of this statistic is that all neighboring nodes are valued equally.



Graph-level Node Degree

The definition of node-level centrality can be extended to the entire graph, in which case we refer to **graph centralization**.

Let v^* be the node with the highest degree centrality in G. Let $X := (Y, Z)$ be a connected graph with $|Y|$ nodes that maximizes the following quantity (with y^* as the node with the highest degree centrality in X):

$$H = \sum_{j=1}^{|Y|} [C_D(y^*) - C_D(y_j)]$$

Accordingly, the **degree centralization** of graph G is given by:

$$C_D(G) = \frac{\sum_{i=1}^{|V|} [C_D(v^*) - C_D(v_i)]}{H}$$

Graph-level Node Degree

$$H = (n-1) \cdot ((n-1)-1) = n^2 - 3n + 2.$$

Thus, for any graph $G := (V, E)$, the degree centralization is given by:

$$C_D(G) = \frac{\sum_{i=1}^{|V|} [C_D(v^*) - C_D(v_i)]}{|V|^2 - 3|V| + 2}$$

Centrality

The node degree does not allow us to understand the **importance** of a node in the graph context.

To better understand the significance of a node in a graph, we can look at its **centrality measure**.

There are different ways to evaluate this statistic:

- From the perspective of eigenvectors
- From the perspective of neighborhood
- From the perspective of closeness

Eigenvector Centrality

Definition: Centrality is defined as the sum of the centralities of all neighbors.

The definition is recursive—this is not very convenient, so an analytical approach is needed.

$$c_u = \frac{1}{\lambda} \sum_{v \in V} A[u, v] c_v \quad \forall u \in V;$$

Eigenvector Centrality

Suppose $n = 3$ and

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Eigenvector Centrality

Then eigenvector centrality (with the normalization $\sum_{i \in N} c_i = 1$) is defined as the solution to the system of equations:

$$\lambda c_1 = c_2$$
$$\lambda c_2 = c_1$$
$$\lambda c_3 = c_1 + c_2$$
$$c_1 + c_2 + c_3 = 1.$$

Solving this system gives:

$$\lambda = 1, \quad c_1 = c_2 = \frac{1}{4}, \quad c_3 = \frac{1}{2}.$$

Eigenvector Centrality

If we define e as the vector of node centralities, we can transition to the standard eigenvalue equation for the adjacency matrix.

Thus, e is an eigenvector of the adjacency matrix A, where:

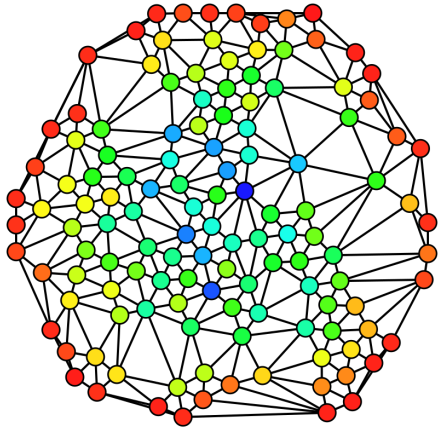
$$\lambda e = Ae$$

Betweenness Centrality

We can assume that a node v is important if it lies on a large number of shortest paths between other nodes that include this node.

$$c_v = \sum_{s \neq v \neq t} \frac{\text{number of shortest paths between } s \text{ and } t \text{ containing } v}{\text{number of shortest paths between } s \text{ and } t}$$

Betweenness Centrality



Closeness Centrality

We can assume that a node is important if it is very close to all other nodes.

$$c_v = \frac{1}{\sum_{u \neq v} \text{shortest path length between } u \text{ and } v}$$

Harmonic Centrality

In a (not necessarily connected) graph, **harmonic centrality** modifies the closeness centrality measure by applying summation and inversion:

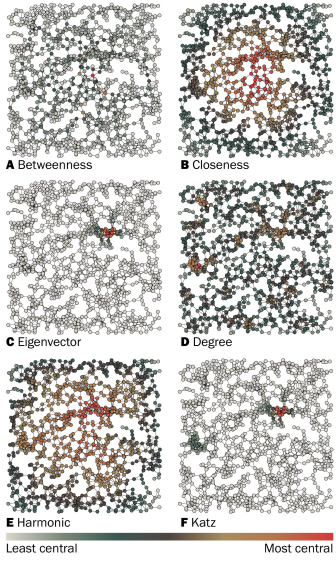
$$H(x) = \sum_{y \neq x} \frac{1}{d(y, x)}$$

Clustering Coefficient

To identify communities, we can compute a statistic for a node that measures the connectivity among all of its neighbors.

$$c_u = \frac{|(v_1, v_2) \in \mathcal{E} : v_1, v_2 \in \mathcal{N}(u)|}{\binom{d_u}{2}}$$

Centrality


A Betweenness
B Closeness
C Eigenvector
D Degree
E Harmonic
F Katz
Least central
Most central

From Clustering Coefficient to Graphlets

If we observe carefully, the clustering coefficient counts the number of *"triangles."*

This raises a natural question—what if we count not only triangles?

Graphlets

Graphlets are subgraphs that describe the structural network of a node's neighbors.

Essentially, they generalize previous feature description methods:

- The **node degree** represents the number of edges connected to a node.
- The **clustering coefficient** represents the number of triangles connected to a node.

Graphlets

There are 73 different graphlets of size 2–5 (while there are 30 non-isomorphic graphs).

Thus, the **graphlet vector** is a 73-dimensional vector that represents how many times a node is part of each specific graphlet. (Of course, the size is not limited to five.)

The meaning of **graphlet degree** is that it provides additional information about the **local topology**.

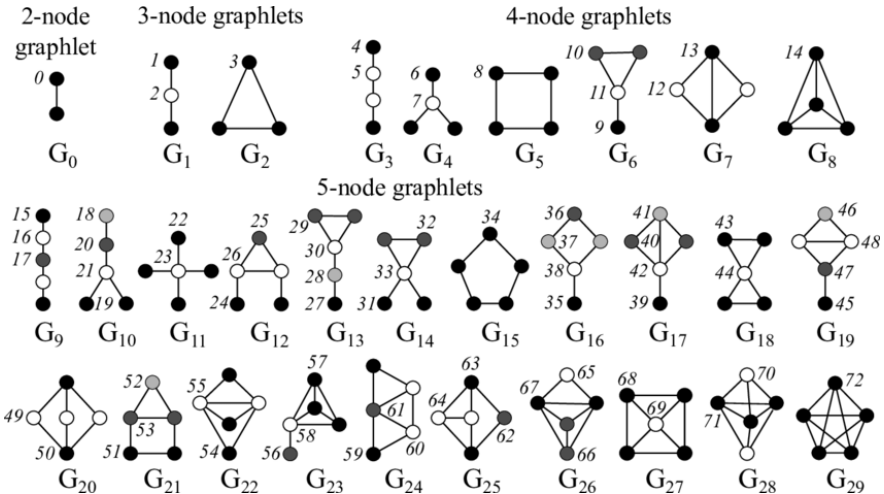
Graphlets

Definition: An **induced subgraph** is a subset of vertices and edges of a graph where the vertices are connected if they are connected in the original graph.

Definition: Isomorphic graphs—Two graphs are called isomorphic if they have the same number of vertices connected in the same way.

Definition: Graphlets—Connected, non-isomorphic induced subgraphs.

Graphlets


2-node graphlet
3-node graphlets
4-node graphlets
5-node graphlets
G0
G1
G2
G3
G4
G5
G6
G7
G8
G9
G10
G11
G12
G13
G14
G15
G16
G17
G18
G19
G20
G21
G22
G23
G24
G25
G26
G27
G28
G29

Predicting Missing Links

Given: A set of edges
Output: New edges

How can we design features for a pair of nodes?

Predicting Missing Links: Two Types of Problems

Random Edge Removal:

- Some nodes are removed from the graph.
- The goal is to predict their reappearance.

Edges Over Time:

- Given timestamps t_0 and t_0' for the graph.
- Generate a ranked list of nodes for timestamps t_1 and t_1'.

Predicting Missing Links

The main idea follows this approach:

- Compute a **score** for each pair of nodes in the graph.
- Rank the pairs.
- Predict the **top** n as new links.
- Check which links actually appeared in the final graph states at t_1 and t_2'.

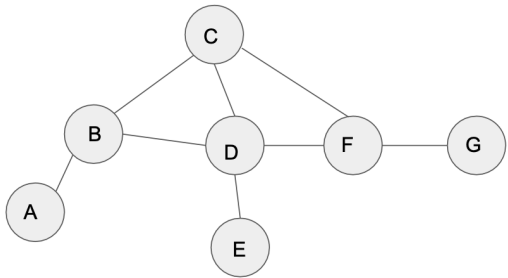
Predicting Missing Links: Features

We consider the following types of features:

- Distance
- Local overlap
- Global overlap

Distance

The distance between a pair of nodes can be defined as the length of the shortest path between them.



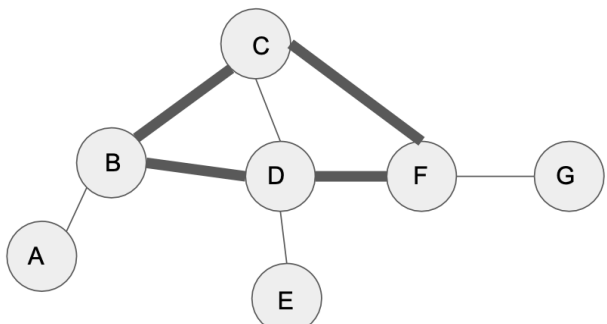
AC = CE = 2

BF = 2

BG = 3

Distance

The shortest paths BE and BF are equal, but from B to F, there exist two shortest paths. This provides additional information that can be extracted.



Local Overlap

It makes sense to analyze the sets of neighbors of two nodes.

Common Neighbors:

$$S[u, v] = |\mathcal{N}(u) \cap \mathcal{N}(v)|$$

Jaccard Coefficient:

$$S_{\text{Jaccard}}[u, v] = \frac{|\mathcal{N}(u) \cap \mathcal{N}(v)|}{|\mathcal{N}(u) \cup \mathcal{N}(v)|}$$

Adamic-Adar Index:

$$S_{\text{AA}}[v_1, v_2] = \sum_{u \in \mathcal{N}(v_1) \cap \mathcal{N}(v_2)} \frac{1}{\log(d_u)}$$

Problems with Local Overlap

If two nodes do not share common neighbors, the selected local overlap metric will be zero.

This makes sense in the context of the current graph state, but as the graph evolves, these nodes might become connected over time. For a more comprehensive understanding, it is necessary to analyze the entire graph structure.

Global Overlap

What makes sense to analyze in an arbitrary graph if we want to evaluate the connection between two distant nodes?

A natural approach is to count all possible paths between two nodes. This leads to the **Katz Index**:

$$S_{\text{Katz}}[u, v] = \sum_{i=1}^{\infty} \beta^i A^i[u, v]$$

At the Graph Level

After analyzing features that characterize nodes and edges, it is time to explore how we can characterize an entire graph.

One option is to simply aggregate node statistics (which, in theory, is a good starting point).

Another approach is to use **graph kernels**. Once a kernel is defined, we can apply classical machine learning techniques, such as **kernel SVM**.

- **Graphlet Kernels**
- **Weisfeiler-Lehman Kernel**

Graphlet Kernels

The main idea is to borrow the concept of the "bag of words" model, replacing it with a **bag of nodes**.

To construct a **graphlet kernel**, we assume the following refinements:

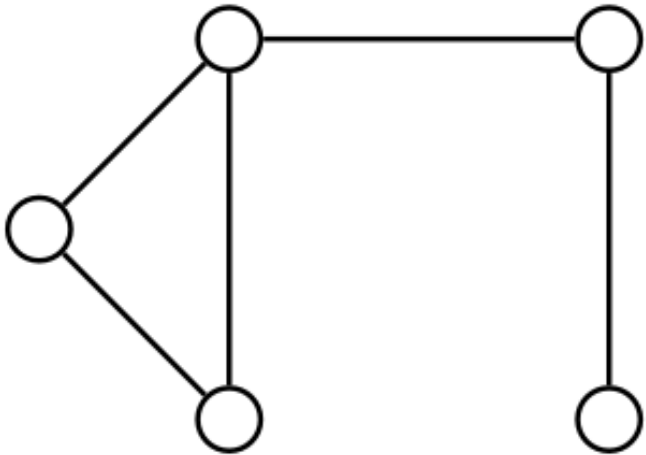
- Nodes in graphlets should not be connected.
- There is no root node.

Graphlet Kernels

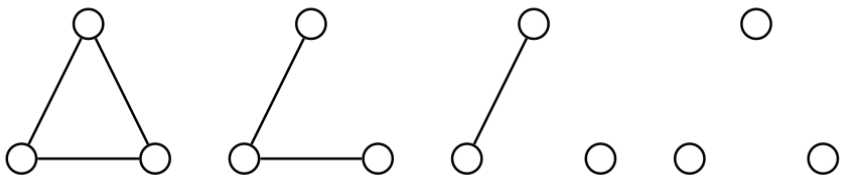
The **graphlet count vector** is a vector that represents the number of graphlets in a network.

$$K(G, G') = f_G^T f_{G'}$$

Graphlet Kernels



Graphlet Kernels



Graphlet Kernels

When comparing different graphs G and G', the result can be inconsistent.

Normalization is required:

$$h_G = \frac{f_G}{\text{Sum}(f_G)}$$

$$K(G, G') = h_G^T h_{G'}$$

Graphlet Kernels: Problems

Very expensive.

For a graph of size n and subgraph size k, the complexity is $**n^k**$.

Why? Because subgraph isomorphism is **NP-hard**.

If the degree of a node is bounded by d, the complexity reduces to $**nd^k**$.

Weisfeiler-Lehman Kernel

The core idea of the algorithm is **iterative neighborhood aggregation**.

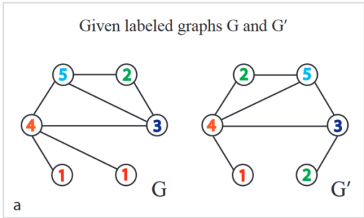
Algorithm:

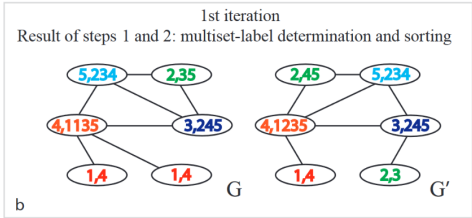
1. Assign a **label** to each node.
2. Update labels iteratively:

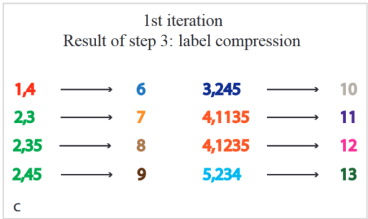
$$l^{(i)}(v) = \text{HASH}\left(\left\{l^{(i-1)}(u) \forall u \in \mathcal{N}(v)\right\}\right)$$

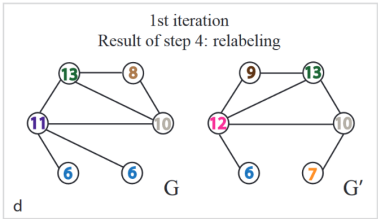
3. After k steps, this vector summarizes the structure of the k-**hop neighborhood**.

Weisfeiler-Lehman Kernel



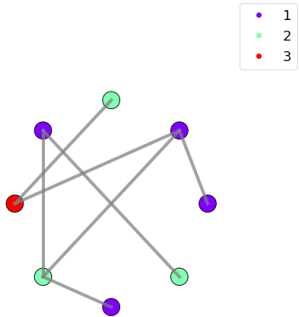




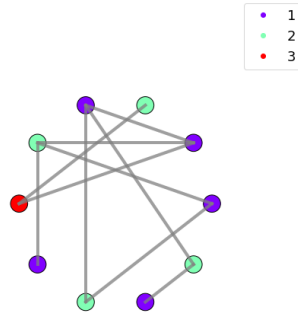


Weisfeiler-Lehman Kernel

Two labeled graphs

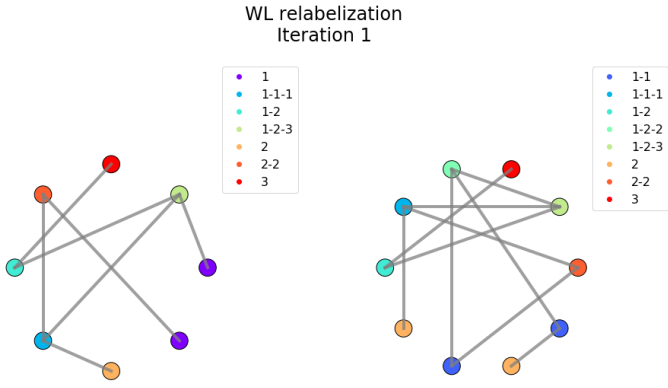


$\phi_0(G) = (4 \quad 3 \quad 1)$

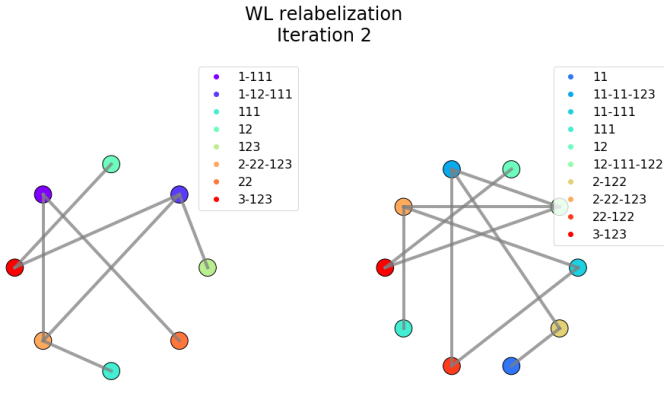


$\phi_0(G) = (5 \quad 4 \quad 1)$

Weisfeiler-Lehman Kernel



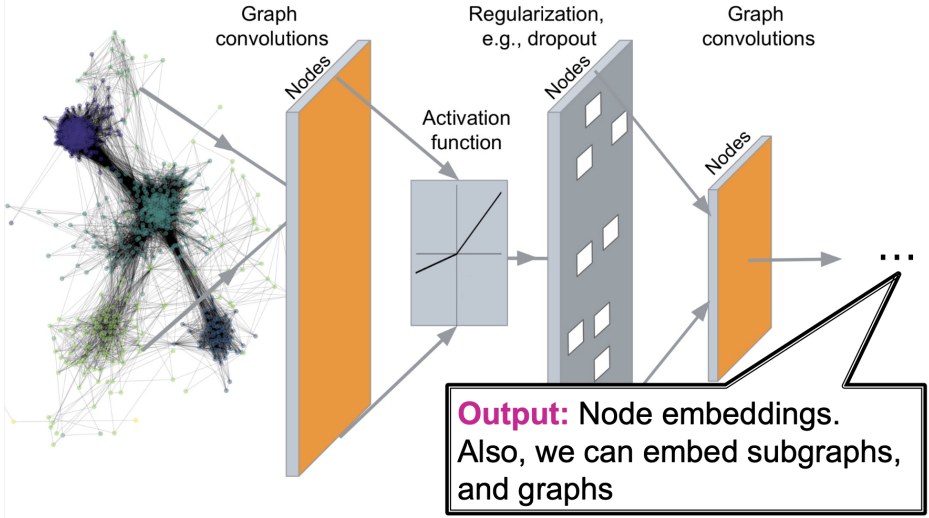
Weisfeiler-Lehman Kernel



WL Kernel

- A **computationally efficient** method.
- When computing the kernel, it is sufficient to track **only the colors** appearing in both graphs.
- Counting colors forms a **sequence**.
- In general, this results in a **sequence over edges**.

Deep Graph Encoders

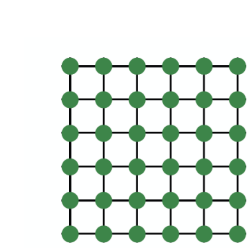


Tasks on Networks

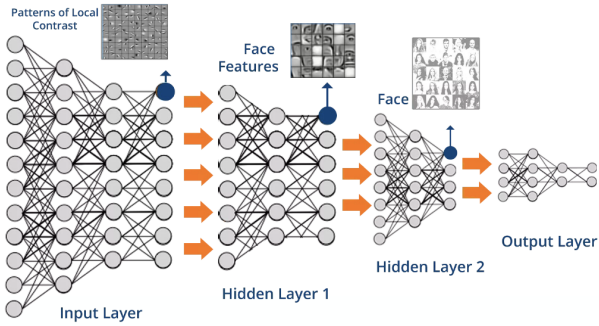
Tasks we will be able to solve:

- **Node classification**
  - Predict the type of a given node
- **Link prediction**
  - Predict whether two nodes are linked
- **Community detection**
  - Identify densely linked clusters of nodes
- **Network similarity**
  - How similar are two (sub)networks

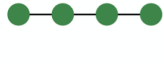
Modern ML Toolbox

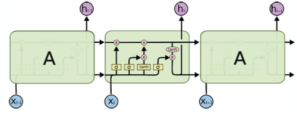


Images



Text/Speech



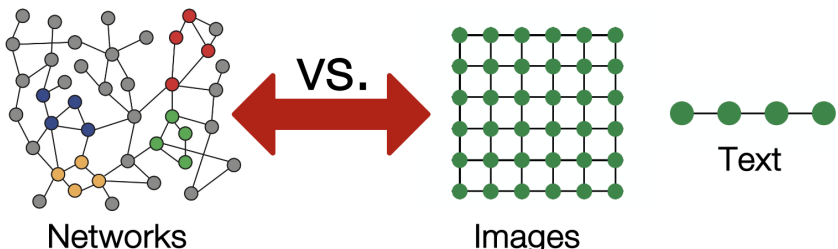


Modern deep learning toolbox is designed for simple sequences & grids

Why is it Hard?

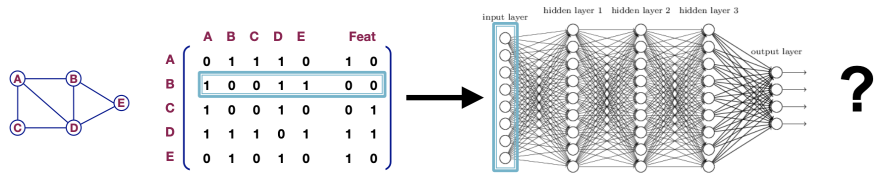
But networks are far more complex!

- Arbitrary size and complex topological structure (i.e., no spatial locality like grids)
- No fixed node ordering or reference point
- Often dynamic and have multimodal features



A Naive Approach

- Join adjacency matrix and features
- Feed them into a deep neural net:

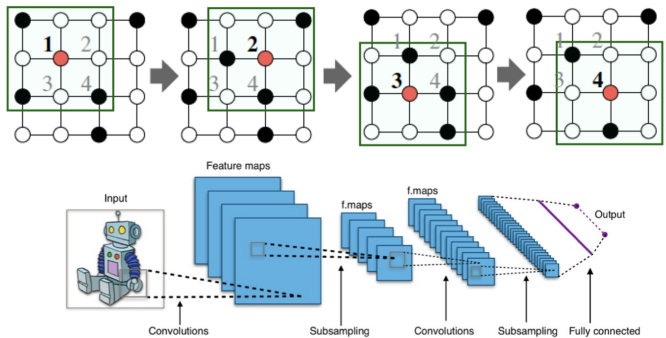


Issues with this idea:

- $O(|V|)$ parameters
- Not applicable to graphs of different sizes
- Sensitive to node ordering

Idea: Convolutional Networks

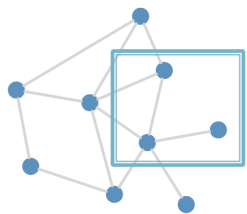
CNN on an image:



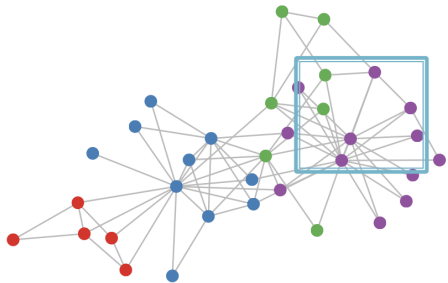
Goal is to generalize convolutions beyond simple lattices.
Leverage node features/attributes (e.g., text, images).

Real-World Graphs

But our graphs look like this:



or this:



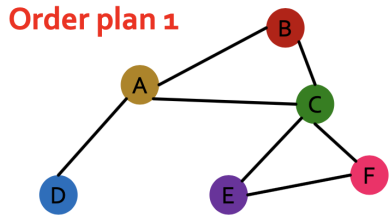
- There is no fixed notion of locality or sliding window on the graph
- Graph is **permutation invariant**

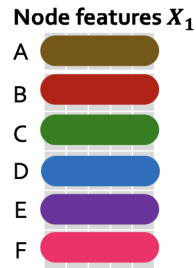
Permutation Invariance

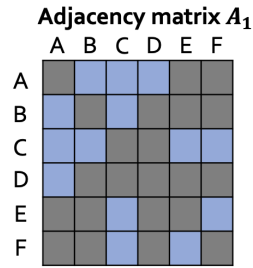
- **Consider we want to embed an entire graph**
- **Observation:** A graph does not have a canonical ordering of its nodes
  - We can have many different node orderings of the same graph.
- **What do we want:** If we learn an embedding function over a graph, we should get the same result (the same embedding) regardless of how the nodes are numbered.

Permutation Invariance

Graph does not have a canonical ordering of the nodes!

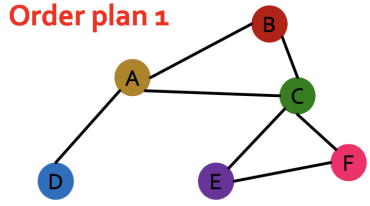


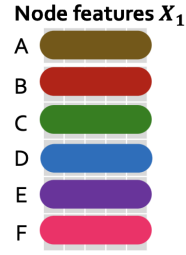


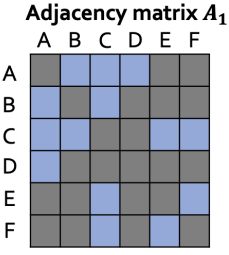


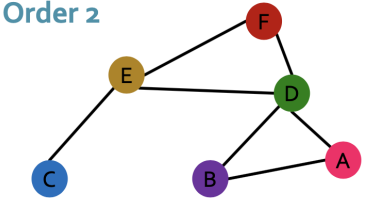
Permutation Invariance

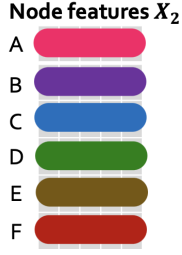
Graph does not have a canonical ordering of the nodes!

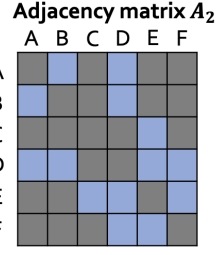












Permutation Invariance

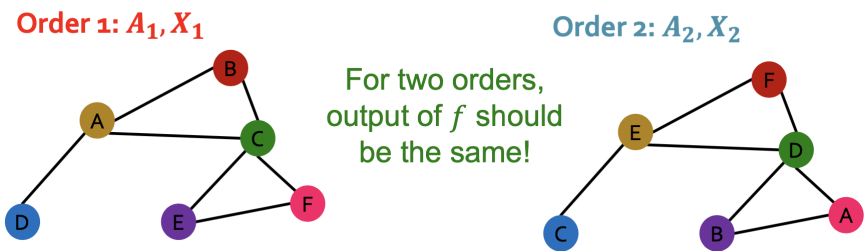
Learned graph representation should be the same for Order 1 and Order 2

What do we mean by "graph representation is the same for two orderings"?

- Consider we learn a function f that maps a graph $G = (A, \mathbf{X})$ to a vector $\mathbb{R}^d$ then:

$$f(A_1, \mathbf{X}_1) = f(A_2, \mathbf{X}_2)$$

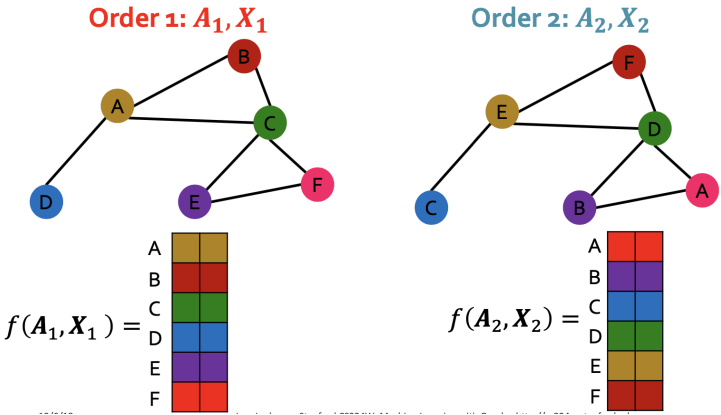
In other words, f maps a graph to a d-dim embedding.
A is the adjacency matrix, $\mathbf{X}$ is the node feature matrix.



Permutation Equivariance

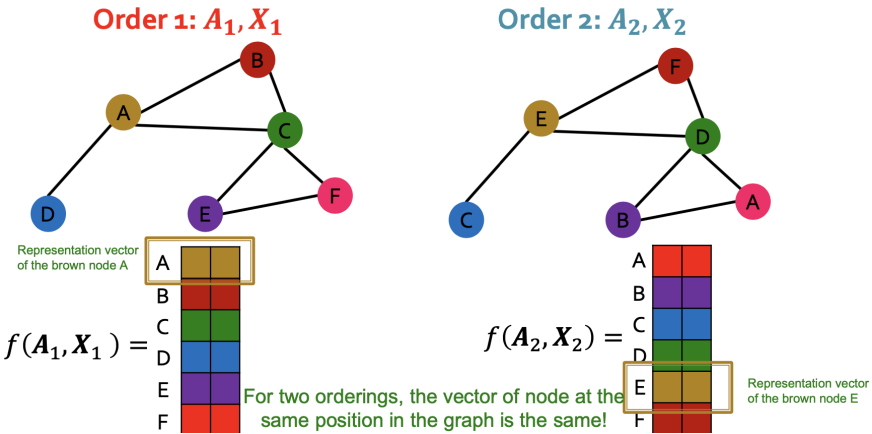
For node representation: We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d}$.

In other words, each node in V is mapped to a d-dim embedding.



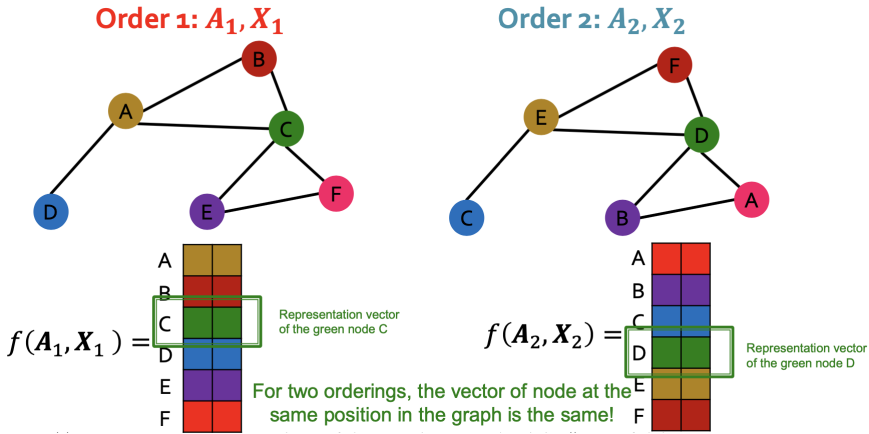
Permutation Equivariance

For node representation: We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d}$.



Permutation Equivariance

For node representation: We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d}$.



Permutation Equivariance

For node representation:

- Consider we learn a function f that maps a graph $G = (A, \mathbf{X})$ to a matrix $\mathbb{R}^{|V| \times d}$.
- If the output vector of a node at the same position in the graph remains unchanged for any ordering, we say f is permutation equivariant.

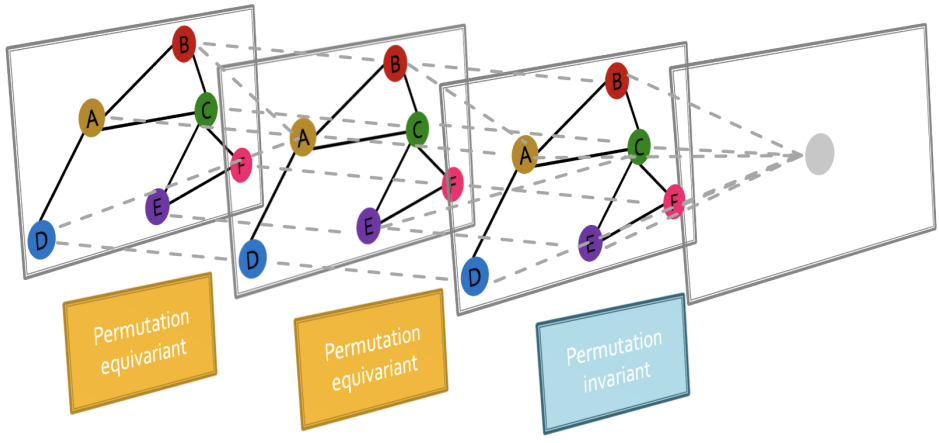
Definition: For any node function $f : \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times m} \to \mathbb{R}^{|V| \times d}$, f is permutation-equivariant if

$$Pf(A, \mathbf{X}) = f(PAP^T, P\mathbf{X})$$

for any permutation P.
f maps each node in V to a d-dim embedding.

Graph Neural Network Overview

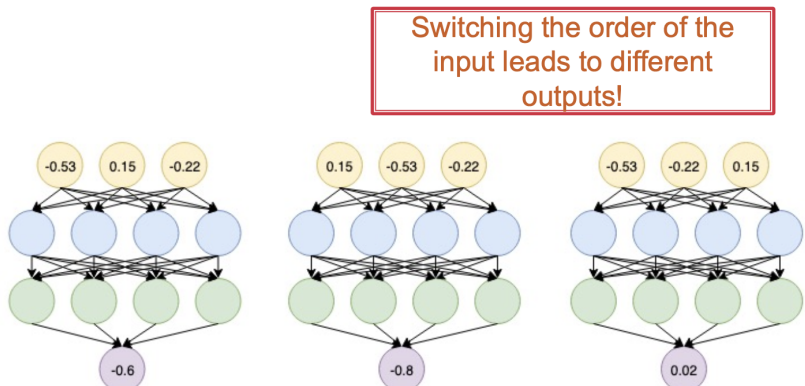
Graph neural networks consist of multiple permutation equivariant / invariant functions.



Graph Neural Network Overview

Are other neural network architectures, e.g., MLPs, permutation invariant / equivariant?

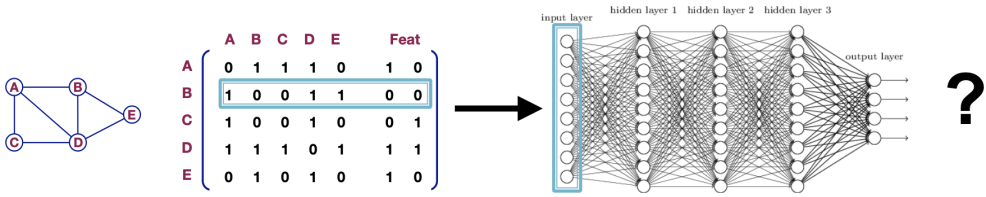
No.



Graph Neural Network Overview

Are other neural network architectures, e.g., MLPs, permutation invariant / equivariant?

No.



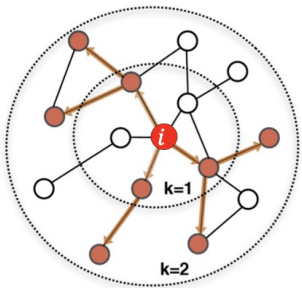
This explains why the naïve MLP approach fails for graphs!

Graph Neural Network Overview

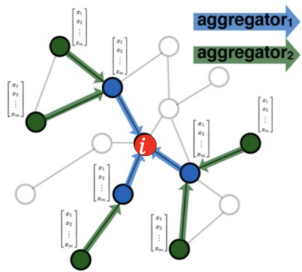
Next: Design graph neural networks that are permutation invariant / equivariant by passing and aggregating information from neighbors!

Graph Convolutional Networks

Idea: Node's neighborhood defines a computation graph



Determine node computation graph

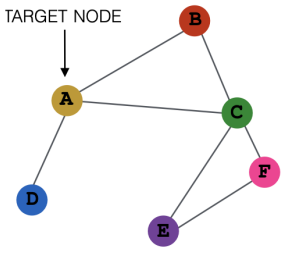


Propagate and transform information

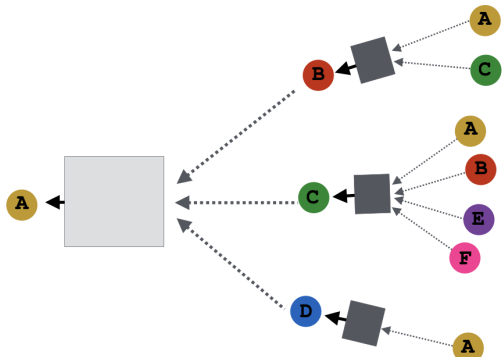
Learn how to propagate information across the graph to compute node features.

Idea: Aggregate Neighbors

Key idea: Generate node embeddings based on local network neighborhoods.



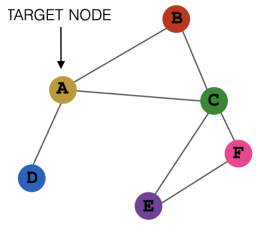
INPUT GRAPH

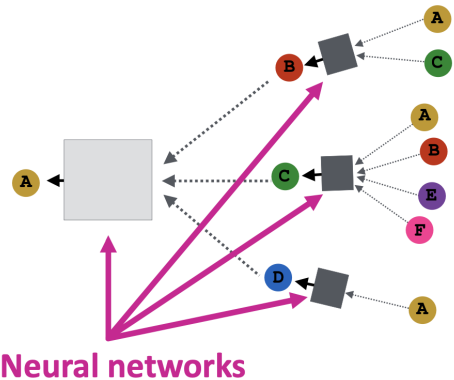


Idea: Aggregate Neighbors

Intuition: Nodes aggregate information from their neighbors using neural networks.

- **Intuition:** Nodes aggregate information from their neighbors using neural networks





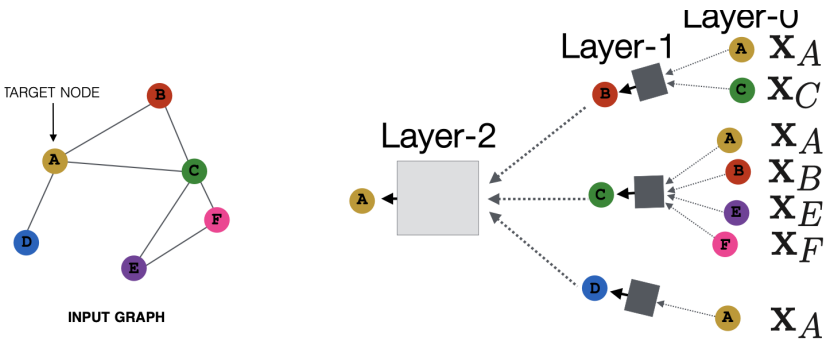
Idea: Aggregate Neighbors

Intuition: Network neighborhood defines a computation graph.
Every node defines a computation graph based on its neighborhood!

Deep Model: Many Layers

Model can be of arbitrary depth:

- Nodes have embeddings at each layer.
- Layer-0 embedding of node v is its input feature, x_v.
- Layer-k embedding gets information from nodes that are k hops away.



The Math: Deep Encoder of a GCN

Basic approach: Average neighbor messages and apply a neural network

- Initial 0-th layer embeddings are equal to node features: $h_v^0 = x_v$
- Update rule:

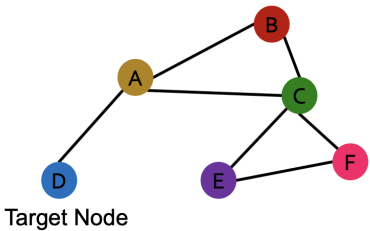
$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \quad \forall k \in \{0, ..., K-1\}$$

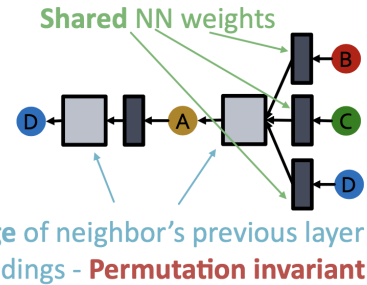
- **Final embedding:** $z_v = h_v^{(K)}$

GCN: Invariance and Equivariance

What are the invariance and equivariance properties for a GCN?

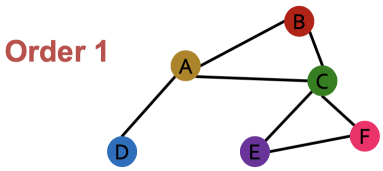
- **Given a node**, the GCN that computes its embedding is **permutation invariant**

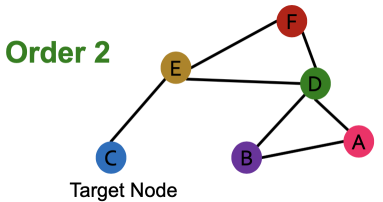


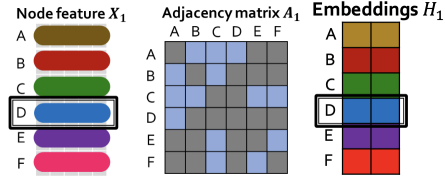


GCN: Invariance and Equivariance

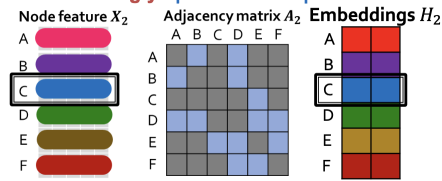
Considering all nodes in a graph, GCN computation is **permutation equivariant**.







Permute the input, the output also permutes accordingly - permutation equivariant



GCN: Invariance and Equivariance

Considering all nodes in a graph, GCN computation is **permutation equivariant**.

Detailed reasoning:

1. The rows of **input node features** and **output embeddings** are **aligned**.
2. We know computing the embedding of a **given node** with GCN is **invariant**.
3. So, after permutation, the **location** of a **given node** in the **input node feature** matrix is changed, and the **output embedding of a given node stays the same** (the colors of node feature and embedding are matched).

This is permutation equivariant.

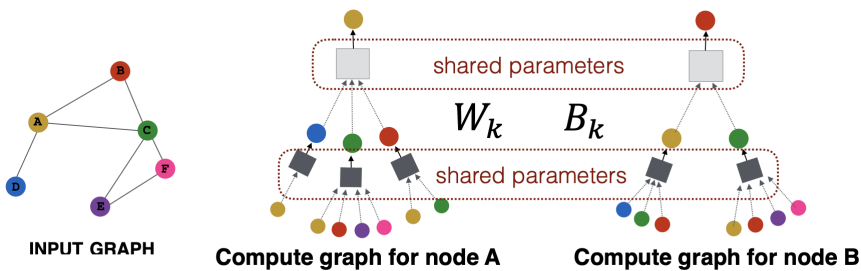
Model Parameters

$$
\begin{aligned}
h_v^{(0)} &= x_v \\
h_v^{(k+1)} &= \sigma\Big(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\Big) \quad \forall k \in \{0, \ldots, K-1\} \\
z_v &= h_v^{(K)}
\end{aligned}
$$

Inductive Capability

The same aggregation parameters are shared for all nodes:

- The number of model parameters is sublinear in $|V|$ and we can generalize to unseen nodes!

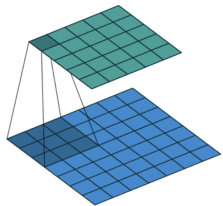


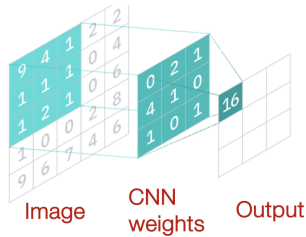
Architecture Comparison

- **How do GNNs compare to prominent architectures such as Convolutional Neural Nets?**

Convolutional Neural Network

Convolutional neural network (CNN) layer with 3x3 filter:





CNN formulation:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in N(v) \cup \{v\}} W_l^u h_u^{(l)}\right), \quad \forall l \in \{0, \ldots, L-1\} \tag{1}$$

N(v) represents the 8 neighbor pixels of v.

GNN vs. CNN

GNN formulation:

$$h_v^{(l+1)} = \sigma\left(\mathbf{W_l} \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}\right), \quad \forall l \in \{0, \ldots, L-1\} \tag{2}$$

CNN formulation (previous slide):

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in N(v) \cup \{v\}} W_l^u h_u^{(l)}\right) \tag{3}$$

If we rewrite:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W_l^u} h_u^{(l)} + B_l h_v^{(l)}\right) \tag{4}$$

GNN vs. CNN

GNN formulation:

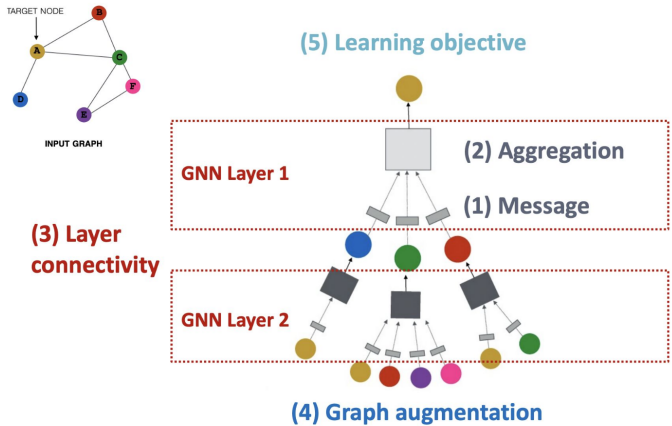
$$h_v^{(l+1)} = \sigma\left(\mathbf{W_l} \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}\right), \quad \forall l \in \{0, \ldots, L-1\} \tag{5}$$

CNN formulation:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W_l^u} h_u^{(l)} + B_l h_v^{(l)}\right) \tag{6}$$

Key difference: We can learn different $\mathbf{W_l^u}$ for different "neighbor" u for pixel v on the image. The reason is we can pick an order for the 9 neighbors using **relative position** to the center pixel: {(-1,-1), (-1,0), (-1,1), ..., (1,1)}

Parts of GNN


TARGET NODE
B
A
C
F
D
E
INPUT GRAPH
(5) Learning objective
(2) Aggregation
GNN Layer 1
(1) Message
(3) Layer connectivity
GNN Layer 2
(4) Graph augmentation

Problems with the First Two Steps

The information at the top node might be lost. It must be preserved.

1)

$$\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$$

$$\mathbf{m}_v^{(l)} = \mathbf{B}^{(l)}\mathbf{h}_v^{(l-1)}$$

2)

$$\mathbf{h}_v^{(l)} = \text{CONCAT}\left(\text{AGG}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right), \mathbf{m}_v^{(l)}\right)$$

GCN (Graph Convolutional Network)

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}\right)$$

$$\mathbf{m}_u^{(l)} = \frac{1}{|N(v)|}\mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$$

$$\mathbf{h}_v^{(l)} = \sigma\left(\text{Sum}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right)\right)$$

GraphSAGE

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \text{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)\right)\right)$$

1) Inside AGG

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)$$

2) Full update

$$\mathbf{h}_v^{(l)} \leftarrow \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)}\right)\right)$$

GraphSAGE AGG

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$

$$\text{AGG} = \text{Mean}\left(\left\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\right\}\right)$$

$$\text{AGG} = \text{LSTM}\left(\left[\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))\right]\right)$$

GAT (Graph Attention Network)

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

$$\alpha_{vu} = \frac{1}{|N(v)|}$$

Issues:

- In GCN/GraphSAGE, α_{vu} is uniform.
- The problem: all nodes are equally important.

Attention Score Calculation

$$e_{vu} = a\left(\mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_v^{(l-1)}\right)$$

Attention Score Computation

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

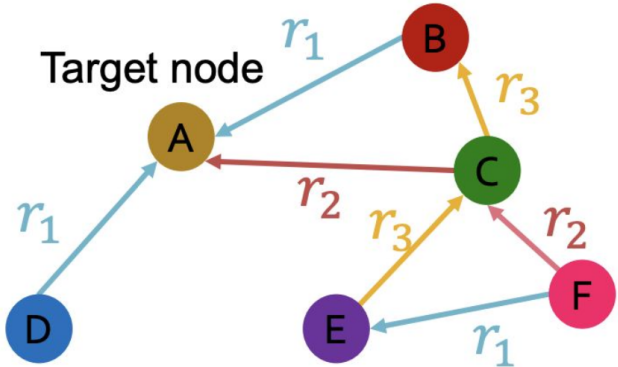
Weighted sum based on the **final attention weight**

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Weighted sum using α_{AB}, α_{AC}, α_{AD}:

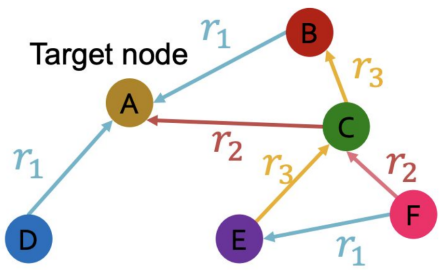
$$\mathbf{h}_A^{(l)} = \sigma\left(\alpha_{AB}\mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)} + \alpha_{AC}\mathbf{W}^{(l)}\mathbf{h}_C^{(l-1)} + \alpha_{AD}\mathbf{W}^{(l)}\mathbf{h}_D^{(l-1)}\right)$$

Knowledge Graph



Input graph

Message Layers



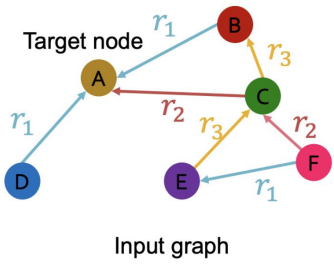
Input graph

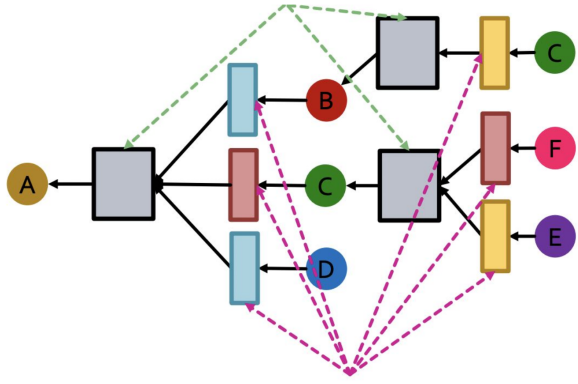
Weights $\mathbf{W}_{r_1}$ for r_1

Weights $\mathbf{W}_{r_2}$ for r_2

Weights $\mathbf{W}_{r_3}$ for r_3

Computational Graph in a Knowledge Graph





RGCN

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{r\in R}\sum_{u\in N_v^r}\frac{1}{c_{v,r}}\mathbf{W}_r^{(l)}\mathbf{h}_u^{(l)} + \mathbf{W}_0^{(l)}\mathbf{h}_v^{(l)}\right)$$

Message

$$\mathbf{m}_{u,r}^{(l)} = \frac{1}{c_{v,r}}\mathbf{W}_r^{(l)}\mathbf{h}_u^{(l)}$$

$$\mathbf{m}_v^{(l)} = \mathbf{W}_0^{(l)}\mathbf{h}_v^{(l)}$$

Aggregation

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathrm{Sum}\left(\left\{\mathbf{m}_{u,r}^{(l)} \mid u \in N(v)\right\} \cup \left\{\mathbf{m}_v^{(l)}\right\}\right)\right)$$

Where Graph Models Win I: Traffic Forecasting

Can a graph model beat a standard recurrent model?

Yes. Traffic sensors form a spatial graph, while LSTM mostly models temporal dynamics.

Method	Architecture	PEMS04	PEMS07	PEMS08	HZMetro	SHMetro
LSTM	recurrent NN, no graph	26.91	30.24	21.49	55.57	59.90
DCRNN	diffusion GNN + RNN	23.70	24.94	18.35	34.50	34.79
Graph WaveNet	graph convolution + temporal conv	20.11	22.24	14.89	31.35	33.31
AGCRN	adaptive graph convolution + RNN	19.76	20.92	16.31	42.35	36.13
STGCN	spatio-temporal GCN	24.58	29.34	19.52	35.70	34.78
ASTGNN	attention-based spatio-temporal GNN	18.32	19.23	12.96	31.14	29.72
TDMGCN	Transformer + multi-graph GCN	**18.18**	**18.66**	**12.86**	**30.92**	**29.36**

Values from Zhang et al., Scientific Reports 2025 [**?**, Tables 3–4].

Where Graph Models Win II: Recommender Systems

Can graph recommenders beat autoencoder-based collaborative filtering?

Yes. Recommendation is naturally link prediction on a user–item graph.

		Yelp		Gowalla		Amazon-Book	
Method	Architecture	R@20	N@20	R@20	N@20	R@20	N@20
AutoR	autoencoder CF, no explicit graph propagation	0.0348	0.0236	0.1298	0.1178	0.0287	0.0156
NGCF	neural graph collaborative filtering	0.0579	0.0477	0.1570	0.1327	0.0344	0.0263
LightGCN	linear graph propagation	0.0653	0.0532	0.1820	0.1547	0.0411	0.0318
SGL	self-supervised graph CF	0.0675	0.0555	0.1709	0.1529	0.0478	0.0379
SHT	Transformer + hypergraph NN	0.0651	0.0546	0.1232	0.0731	0.0740	0.0553
TransGNN	Transformer + GNN	**0.0817**	**0.0734**	**0.1887**	**0.1602**	**0.0801**	**0.0603**

R@20 = Recall@20, N@20 = NDCG@20. Values from Zhang et al., SIGIR 2024 [?, Table 2].

Where Graph Models Win III: Molecules vs SMILES-CNN

Can a molecular graph model beat a CNN over SMILES strings?

Yes. Molecules are graphs of atoms and bonds, not just character sequences.

Method	Drug encoder	RMSE ↓	PCC ↑	R^2 ↑
tCNN	CNN over SMILES string	0.056	0.356	0.027
GraphDRP-GCN	molecular GCN	0.063	0.450	0.153
GraphDRP-GAT	molecular GAT	0.071	0.351	-0.041
TGSA	GraphSAGE-based drug encoder	2.809	0.329	0.026
XGDP-GCN	molecular GCN + explanations	0.056	0.400	0.048
XGDP-GAT	molecular GAT + explanations	0.053	0.448	0.149
XGDP-GAT_E	molecular GAT with edge features	**0.052**	**0.505**	**0.164**

Values from Wang et al., Scientific Reports 2025 [**?**, Table 2].

Where Graph Models Win IV: Protein Function Prediction

Can graph/structure-aware models beat sequence CNNs?

Yes. Protein function depends on structure, interactions, and local neighborhoods.

		AUPR ↑			Fmax ↑		
Method	Architecture	MF	BP	CC	MF	BP	CC
DeepGO	sequence-based deep CNN model	0.391	0.189	0.258	0.576	0.500	0.589
DeepFRI	structure-based GNN	0.495	0.265	0.274	0.627	0.546	0.617
GAT-GO	graph attention model	0.660	0.381	0.479	0.633	0.492	0.547
HEAL	hierarchical / structure-aware model	0.661	0.339	0.435	0.733	0.613	0.673
TAWFN	CNN + GCN fusion	**0.718**	**0.385**	**0.488**	**0.762**	**0.628**	**0.693**

Values from Meng and Wang, Bioinformatics 2024 [?, Table 1].

DiGress: Discrete Diffusion for Graph Generation

Task and architecture

Task: unconditional or conditional graph generation. Generate graphs with categorical node and edge attributes.
Data: abstract graphs such as SBM and planar graphs; molecular graphs such as QM9, MOSES, and GuacaMol.
Architecture: DiGress starts from a clean graph $G = (X, E)$, corrupts node and edge categories with a discrete Markov noise process, and trains a graph transformer to predict the clean graph from the noisy graph:

$$G_0 \to G_1 \to \cdots \to G_T, \qquad \phi_\theta(G_t, t) \approx G_0.$$

Why graph-specific? Gaussian noise on adjacency matrices destroys sparsity and graph-theoretic structure. DiGress keeps the object discrete: nodes, edges, and edge absence remain categorical states.

DiGress: Benchmark Results

	Stochastic block model				Planar graphs			
Model	Deg ↓	Clus ↓	Orb ↓	V.U.N. ↑	Deg ↓	Clus ↓	Orb ↓	V.U.N. ↑
GraphRNN	6.9	1.7	3.1	5%	24.5	9.0	2508	0%
GRAN	14.1	1.7	2.1	25%	3.5	1.4	1.8	0%
GG-GAN	4.4	2.1	2.3	25%	–	–	–	–
SPECTRE	1.9	1.6	**1.6**	53%	2.5	2.5	2.4	25%
ConGress	34.1	3.1	4.5	0%	23.8	8.8	2590	0%
DiGress	**1.6**	**1.5**	1.7	**74%**	**1.4**	**1.2**	**1.7**	**75%**

Source: Vignac et al., ICLR 2023 [?, Table 1].

SE(3)-Transformer: Equivariant Attention for 3D Graphs

Task and architecture

Task: learning on 3D point clouds and geometric graphs: particle systems, molecular structures, protein structures, and 3D objects.
Input: nodes with features and coordinates:

$$\{(h_i, x_i)\}_{i=1}^{n}, \qquad x_i \in \mathbb{R}^3.$$

Architecture: SE(3)-Transformer modifies self-attention so that attention weights are invariant and value features transform equivariantly under 3D rotations and translations.
Key property: If the input geometry is rotated, the vector-valued outputs rotate in the same way:

$$f(Rx + t) = Rf(x).$$

SE(3)-Transformer: Benchmark Results

Method	Position MSE ↓	Position Δ_{EQ} ↓	Velocity MSE ↓	Velocity Δ_{EQ} ↓
Linear	0.0691	–	0.261	–
DeepSet	0.0639	0.038	0.246	1.11
Tensor Field	0.0151	$1.9 \cdot 10^{-7}$	0.125	$5.0 \cdot 10^{-7}$
Set Transformer	0.0139	0.167	0.101	0.37
SE(3)-Transformer	**0.0076**	$3.2 \cdot 10^{-7}$	**0.075**	$6.3 \cdot 10^{-7}$

Source: Fuchs et al., NeurIPS 2020 [?, Table 1].

MPNN: Message Passing Neural Networks for Molecules

Task and architecture

Task: molecular property prediction. Given a molecular graph, predict quantum-chemical or biological properties of the whole molecule.

Message passing phase:

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{vw}), \qquad h_v^{t+1} = U_t(h_v^t, m_v^{t+1}).$$

Readout phase:

$$\hat{y} = R(\{h_v^T : v \in G\}).$$

Interpretation: after T steps, each atom representation summarizes its T-hop chemical environment; the readout aggregates atom representations into a molecular representation.

MPNN: Benchmark Results on QM9

Method	Representation	μ	HOMO	LUMO	U_0	G	Ω	Avg.
BAML	engineered descriptor	4.34	2.20	2.76	1.21	1.20	0.27	2.17
BOB	bag of bonds	4.23	2.20	2.74	1.43	1.42	0.35	2.08
CM	Coulomb matrix	4.49	3.09	4.26	2.98	2.97	1.32	3.37
ECFP4	fingerprint	4.82	2.89	3.10	85.01	78.36	1.47	53.97
HDAD	projected histograms	3.34	1.54	1.96	0.58	0.59	0.34	1.35
GC	molecular graph conv.	0.70	1.18	1.10	3.02	2.95	0.32	2.59
GG-NN	gated graph NN	1.22	1.17	1.08	0.83	0.78	0.53	1.36
MPNN, enn-s2s	edge network + Set2Set	0.30	0.99	0.87	0.45	0.44	0.19	0.68
MPNN, enn-s2s-ens5	ensemble	**0.20**	**0.74**	**0.65**	**0.33**	**0.34**	**0.15**	**0.52**

Selected targets from Gilmer et al., ICML 2017 [?, Table 2].

Graph Foundation Models and LLM + Graph

Main idea

Classical GNNs are usually trained for one graph and one task. Graph foundation models aim to transfer across graphs, domains, and tasks.

Common ingredients:

- **Graph tokenizer:** converts arbitrary graph structures into transferable tokens.
- **Graph transformer:** captures global topological dependencies.
- **Prompting / instruction tuning:** unifies different node, edge, and graph tasks.
- **LLM interaction:** uses text, labels, prompts, or generated graphs to improve generalization.

Examples

OFA: converts different graph tasks into text-attributed graph classification tasks.
OpenGraph: uses LLM-generated graph data, topology-aware tokenization, and a scalable graph transformer.
GraphGPT: aligns graph encoders with LLM tokens and uses graph instruction tuning.

OpenGraph Results

	Link prediction, Recall@20 ↑			Node classification, Accuracy ↑		
Method / setting	ogbl-ddi	ML-1M	Amazon-book	Cora	Citeseer	Pubmed
MLP, 5-shot	0.0621	0.0851	0.0092	0.3930	0.3690	0.5240
GCN, 5-shot	0.0705	0.1054	0.0251	0.5470	0.4910	0.5090
GAT, 5-shot	0.0711	0.1506	0.0228	0.5850	0.4940	0.5780
GIN, 5-shot	0.0735	0.1458	0.0252	0.5400	0.5210	0.5070
DGI, 5-shot	0.0821	0.1687	0.0300	0.4880	0.4450	0.4890
GraphCL, 5-shot	0.0740	0.1416	0.0270	0.5610	0.4300	0.5230
GPrompt, 5-shot	0.0769	0.1340	0.0287	0.5510	0.5570	0.5130
OpenGraph, 0-shot	**0.0921**	**0.1911**	**0.0485**	**0.7504**	**0.7221**	**0.6869**

Selected results from Xia, Kao, and Huang, Findings of EMNLP 2024 [?, Table 1].

GraphGPT Results for LLM + Graph

Method	Arxiv → Arxiv	Arxiv → PubMed	Arxiv → Cora	Arxiv+PubMed → Cora	Arxiv+PubMed → Arxiv
MLP	0.5179	0.3940	0.0258	0.0220	0.2127
GraphSAGE	0.5480	0.3950	0.0328	0.0132	0.1281
GCN	0.5267	0.3940	0.0214	0.0187	0.0122
GAT	0.5332	0.3940	0.0167	0.0161	0.1707
NodeFormer	0.5922	0.2064	0.0152	0.0144	0.2713
DIFFormer	0.5986	0.2959	0.0161	0.0100	0.1637
Vicuna-7B-v1.5	0.4962	0.6351	0.1489	0.1489	0.4962
GraphGPT-stage2	**0.7511**	0.6484	0.0813	0.0934	0.6278
GraphGPT-std	0.6258	**0.7011**	0.1256	0.1501	0.6390
GraphGPT-cot	0.5759	0.5213	**0.1813**	**0.1647**	**0.6476**

Selected accuracy results from Tang et al., SIGIR 2024 [**?**, Table 1].

The End